

## Scenario-Based Interview Questions — MBPT Project + DRF

These simulate "what would you do if..." questions — very common for 3-YOE roles because they test judgment, not memorization. Some scenarios below are framed as **hypotheticals you reason through live**, not claims that they actually happened to you. I've marked which is which — don't blur the line in the interview, since "here's how I'd approach it" is a completely legitimate and expected answer.

---

### Category 1: Data Consistency & Concurrency

**Scenario 1: "Two forwarder systems send status updates for the same HU at nearly the same time. What happens, and how would you prevent a race condition?"**

**How to answer (reasoning-based, not a claimed incident):** *"In my actual implementation, each request does a read-then-write on the HandlingUnit — fetch the object, check status, update, save. If two requests hit concurrently, there's a real risk of a lost update: both read 'in transit', both write, and one overwrite silently wins. The safe fix is one of:*

1. `select_for_update()` inside a `transaction.atomic()` block to lock the row for the duration of the update — the second request blocks until the first commits.
2. Or optimistic locking — add a `version` field, and reject the update if the version doesn't match what was read.

*I'd pick `select_for_update` here because delivery status updates are relatively rare per HU (a handful of times over its lifecycle), so lock contention is low, and pessimistic locking is simpler to reason about than optimistic retry logic for this volume."*

**Follow-up: "What if the request also touches Tracking and Route — do you need to lock those too?"** **A:** *"Yes — ideally the whole cascading update (HU → Tracking → Route → parent Route) happens inside one `transaction.atomic()` block with*

`select_for_update()` on each row being read-then-written, so a concurrent update to the same route doesn't interleave with yours mid-cascade."

---

**Scenario 2: "A driver's phone loses signal mid-delivery and the app retries the same scan-HU request 3 times once connectivity returns. How do you avoid processing it 3 times?"**

**A:** "This is an idempotency problem. The fix is to make the endpoint idempotent — either:

1. Check current state before applying the transition: if the HU is already marked delivered with a matching timestamp/driver, return success without reprocessing.
2. Have the client send an idempotency key (e.g., a UUID generated once per scan event) and store processed keys, rejecting duplicates.

In my Forwarder Data Receiver logic, there's actually a related check already — it skips processing if `hu_obj.status == HU_STATUS.delivered`, which handles the simplest case of 'don't re-deliver something already delivered.' For a stricter guarantee I'd add an idempotency key on top of that, especially for the create-driver-if-not-exists path, since that one isn't naturally idempotent without a uniqueness check."

---

## Category 2: Scaling & Performance

**Scenario 3: "Your forwarder report API starts timing out as the HU table grows to millions of rows. Walk me through how you'd diagnose and fix it."**

**A:** "First, diagnose before fixing — I'd check the query plan with `EXPLAIN ANALYZE` on the actual filter being used. Given the code does `tracking_list__iregex` and `route_list__iregex` matching on comma-separated text fields, that's a strong suspect — regex matching on a TEXT column can't use a standard B-tree index, so it's doing a full table scan.

Fixes, in order of effort:

1. Add a proper join table (many-to-many `HandlingUnit↔Tracking`, `HandlingUnit↔Route`) instead of comma-separated text fields, so lookups become indexed FK joins instead of regex scans. This is the correct long-term fix but requires a migration.
2. Short-term: add a GIN index with `(pg_trgm)` on the text columns to speed up the regex/LIKE matching without a schema redesign.
3. Add pagination and required date-range filters on the report endpoint so it can't be called unbounded.
4. Add `(select_related)`/`(prefetch_related)` for any related object lookups happening in a loop during serialization — I know the report serializer does per-tracking lookups (`Route`, `Dealer`, `Driver`) in a loop, which is an  $N+1$  pattern at scale."

**Why this is a strong answer:** it references the actual `(route_list)`/`(tracking_list)` design in your real models — showing you understand the trade-off your team already made (denormalized text fields for flexibility) and its real cost at scale.

---

**Scenario 4: "The SQS-based data loader starts falling behind — messages pile up faster than they're processed. What do you do?"**

**A:** "First check whether it's a throughput problem or a stuck-consumer problem. If messages are processing but slowly, I'd scale horizontally — run multiple worker instances/tasks polling the same queue, since SQS naturally load-balances across consumers. If a specific message type is slow (e.g., large batch payloads), I'd profile that path specifically rather than assume it's uniform.

If it's a stuck-consumer problem — a message causing an exception that's silently swallowed and retried forever — I'd check the Dead Letter Queue configuration. Messages should move to a DLQ after  $N$  failed attempts so one poison message doesn't block the whole queue's throughput, and I'd alert on DLQ depth so it's caught immediately rather than discovered days later."

---

### Category 3: Failure & Debugging

**Scenario 5: "A dealer reports their shipment shows 'in transit' in the app, but the driver says he already delivered it yesterday. How do you debug this?"**

**A:** *"I'd work backward from the data:*

- 1. Query the `HandlingUnit(s)` for that dealer's `tracking_id` — check `status`, `delivered_date`, and `delivered_date_list` directly in the DB, not through the API, to rule out a serializer/display bug versus a data bug.*
- 2. Check the Tracking record's status separately — since Route/Tracking/HU statuses are updated in a cascade, it's possible the HU updated but the cascading check to update Tracking didn't fire (e.g., if another HU sharing that tracking point wasn't yet delivered, so `hus_tracking_count` wasn't zero).*
- 3. Check application logs (`db_logger` calls) around the time of the driver's delivery scan for that HU ID — errors during the forwarder-data-receiver call, or a validation rejection that silently dropped the update into the per-HU error list without the driver noticing (since it doesn't hard-fail).*
- 4. If the data's genuinely inconsistent, I'd manually correct it, but more importantly file it as a bug — likely candidates being a validation edge case I hadn't covered, or a partial-batch failure that wasn't surfaced to the forwarder driver's app."*

**Why this works:** it demonstrates you understand your own cascading logic well enough to know exactly which step could silently fail — which is a stronger signal than a generic "I'd check the logs" answer.

---

**Scenario 6: "Your microservice's shared-database approach means the monolith team changes a column name without telling you, and your service starts throwing 500s in production. What's your immediate response, and how do you prevent it next time?"**

**A:** *"Immediate: check error logs/Sentry for the exact exception — likely a `FieldError` or `ProgrammingError` from Django hitting a column that no longer exists — confirm which*

*table/column, and roll back the monolith's migration if possible while a fix is prepared, since that's the fastest way to restore service.*

*Prevention: this is exactly the risk of the shared-database pattern, so the mitigation is process, not just code — the two teams need a contract: schema changes to shared tables get flagged in advance (e.g., in a shared changelog or a required review step), and ideally there's a staging environment where both services run against the same DB before either deploys to prod. Longer-term, this is one of the arguments for eventually moving to fully separate databases with event-driven sync — it removes this class of cross-team breakage entirely."*

---

#### **Category 4: Design & Trade-offs**

**Scenario 7: "You need to add a feature where dealers get notified the moment their shipment is marked delivered. How would you design this without slowing down the delivery API itself?"**

**A:** *"I wouldn't send the notification synchronously inside the request that marks the HU delivered — that couples notification latency/failures to the core delivery-update path, which is the wrong trade-off (a flaky email provider shouldn't be able to fail a delivery scan).*

*Instead: after the DB update commits, publish a lightweight event (`hu.delivered`), with hu\_id/dealer\_id) to a queue — RabbitMQ, since that's already in use for SAP integration, or SQS if staying AWS-native. A separate consumer picks it up and sends the notification (push via Firebase, email, SMS via Retarus — all of which are already integrated). This keeps the API fast and makes notification failures retryable independently without blocking or corrupting the delivery status update itself."*

---

**Scenario 8: "Product wants a new 'Forwarder Analytics Dashboard' with real-time delivery metrics. How would you architect the backend for this?"**

**A:** *"I'd avoid running heavy aggregation queries directly against the same tables the live tracking APIs hit — that risks slowing down production HUI updates for the sake of a dashboard."*

*I'd lean toward CQRS-style separation: keep write-side APIs (like my Forwarder Data Receiver) exactly as they are, and build a read-optimized reporting path — either a materialized view refreshed periodically, or a separate reporting database populated via the same event-publishing pattern from Scenario 7. For 'real-time,' I'd clarify with product what real-time actually means here — sub-second, or 'updated within a minute or two' — since that changes the design significantly (WebSocket push vs. periodic polling of a pre-aggregated table)."*

---

## **Category 5: Security & Reliability**

**Scenario 9: "Your Forwarder Data Receiver API uses `permission_classes = [AllowAny]` since external systems call it without a user login. An interviewer asks: isn't that a security risk?"**

**A:** *"Yes, and it's worth being upfront about that trade-off rather than defending it as ideal. `AllowAny` means anyone who knows the URL can technically call it. In a well-secured version of this, I'd want at minimum an API key or shared secret per forwarder company, validated in the view or via an API Gateway layer before it even reaches Django — Kong, which I know is used elsewhere in this project's architecture, supports exactly this via a plugin. If I were revisiting this today, I'd push for that instead of relying purely on `AllowAny` plus the forwarder-company-name lookup as an implicit access check, since a company name isn't a secret."*

**Why this is a good answer:** admitting a real limitation, with a concrete fix, is far more credible than pretending everything you built is airtight — interviewers specifically probe for this kind of honesty at the 3-YOE level.

---

**Scenario 10:** "How would you roll out a breaking API change (e.g., changing the shape of `hu_details` in the request payload) without breaking every forwarder integration at once?"

**A:** *"Version the endpoint rather than mutating it in place —*

*`/api/v1/forwarder/forwarder-data/` stays exactly as-is, and a new `/api/v2/...` endpoint takes the new payload shape. Both run in parallel; existing forwarder integrations keep working against v1 with zero changes needed on their end. I'd give a deprecation timeline (e.g., 3-6 months), communicate it to each integrated forwarder partner directly since this crosses company boundaries, and only remove v1 once traffic on it drops to zero or the deadline passes."*

---

### **Quick Reference: The Pattern Behind All These Answers**

Notice every answer above follows the same shape:

1. **State the core risk/trade-off in one sentence** (shows you understand *why* it's a problem, not just that it is one)
2. **Give a concrete technical fix**, tied to tools/patterns already in your project where possible
3. **Acknowledge the trade-off of your own fix** if there is one

That third step is what separates a 3-YOE-level answer from a junior one — juniors give "the fix"; people ready for the next level show they know every fix has a cost.